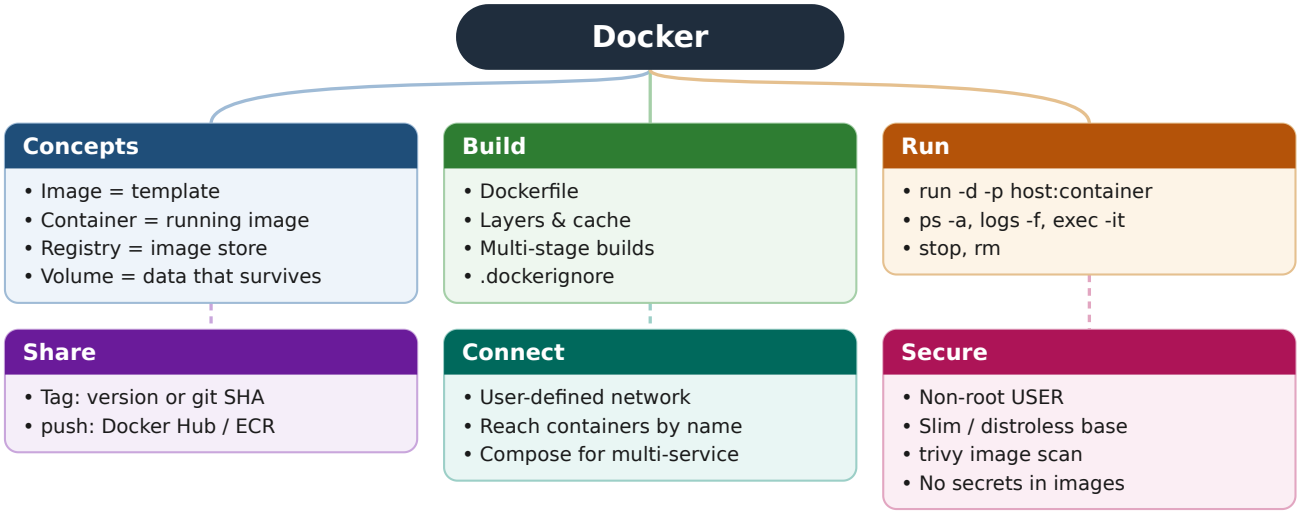
8. Docker

In this section: 8.1 Concepts · 8.2 Core commands · 8.3 Dockerfile · 8.4 Multi-stage build · 8.5 Registries · 8.6 Volumes and storage · 8.7 Networking · 8.8 Docker Compose · 8.9 Container security · 8.10 Container troubleshooting · 8.11 More Docker commands and the container ecosystem



8.1 Concepts

Term	Meaning
Container	An app plus everything it needs, running isolated on a shared OS kernel
Image	A read-only template used to create containers (built from a Dockerfile)
Dockerfile	Text instructions to build an image
Layer	Each instruction creates a cached layer. Unchanged layers are reused
Registry	Stores images: Docker Hub, Amazon ECR, GitHub Container Registry
Tag	Image version label: `myapp:1.2.0`. `latest` is just a default name, not "newest"
Volume	Storage that outlives a container
Docker Engine	The background service (daemon) that builds and runs containers
Namespaces / cgroups	Linux features that isolate containers / limit their CPU and memory

	Virtual machine	Container
Includes	Full guest OS	App + libraries only
Size	GBs	MBs
Start time	Minutes	Seconds
Isolation	Stronger (own kernel)	Shares the host kernel

8.2 Core commands

Command	What it does
`docker version`, `docker info`	Check the installation
`docker run -d -p 8080:80 --name web nginx`	Run in the background (`-d`), map host 8080 → container 80, name it
`docker run -it ubuntu bash`	Interactive shell in a new container
`docker ps` / `docker ps -a`	Running / all containers
`docker logs -f web`	Follow container logs
`docker exec -it web sh`	Open a shell inside a running container
`docker stop web`, `docker start web`, `docker rm web`	Stop / start / delete
`docker images`	List local images
`docker pull nginx:1.27`	Download an image
`docker rmi image`	Delete an image
`docker inspect web`	Full details (IP, mounts, env) as JSON
`docker stats`	Live CPU/memory per container
`docker system df`, `docker system prune`	Disk usage / clean unused data

8.3 Dockerfile

```
FROM python:3.12-slim              # base image (small "slim" variant)
WORKDIR /app                       # working directory inside the image
COPY requirements.txt .            # copy dependency list first (better caching)
RUN pip install --no-cache-dir -r requirements.txt
COPY . .                           # then copy the code
ENV PORT=8000                      # non-secret default config
EXPOSE 8000                        # documents the port
RUN useradd -m appuser
USER appuser                       # don't run as root
HEALTHCHECK CMD python -c \
  "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"
CMD ["python", "app.py"]           # default command when the container starts
```

Instruction	Purpose
`FROM`	Base image
`RUN`	Run a command at **build** time
`COPY` / `ADD`	Copy files in (prefer COPY)
`WORKDIR`	Set the directory
`ENV` / `ARG`	Runtime variable / build-time variable
`EXPOSE`	Document a port
`USER`	User to run as
`CMD`	Default command (easy to override)

Instruction	Purpose
`ENTRYPOINT`	Fixed main command; `CMD` becomes its default arguments

Build and run: `docker build -t myapp:1.0 .` → `docker run -d -p 8000:8000 myapp:1.0`

.dockerignore: `.git`, `.env`, `__pycache__`, `node_modules`, `*.log` (keeps images small and secrets out).

8.4 Multi-stage build

```
FROM golang:1.23 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o app

FROM gcr.io/distroless/static     # tiny final image: no shell, no compiler
COPY --from=build /src/app /app
USER nonroot
ENTRYPOINT ["/app"]
```

Build tools stay in the first stage; only the result is shipped. Smaller image = faster and fewer vulnerabilities.

8.5 Registries

Command	What it does
`docker login`	Log in to Docker Hub
`docker tag myapp:1.0 user/myapp:1.0`	Add a registry name to an image
`docker push user/myapp:1.0`	Upload
`aws ecr get-login-password | docker login --username AWS --password-stdin <acct>.dkr.ecr.<region>.amazonaws.com`	Log in to ECR

Tagging: use versions or Git commit SHAs (`myapp:1.4.2`, `myapp:3f9a1c2`), not only `latest`. Turn on immutable tags in ECR.

8.6 Volumes and storage

Type	Example	Use
Named volume	`-v pgdata:/var/lib/postgresql/data`	Database data, managed by Docker
Bind mount	`-v $(pwd)/src:/app`	Live code during development
tmpfs	`--tmpfs /tmp`	Temporary in-memory data

`docker volume ls`, `docker volume inspect pgdata`, `docker volume rm pgdata`. Container filesystems are temporary: data not in a volume is lost when the container is removed.

8.7 Networking

Network	Behaviour
bridge (default)	Private network on the host; publish ports with `-p`
user-defined bridge	Containers can reach each other **by name** (built-in DNS)
host	Container uses the host's network directly
none	No network

```
docker network create appnet
# the password is kept in db.env, not typed in the command
docker run -d --name db  --network appnet --env-file db.env postgres:16
docker run -d --name api --network appnet -p 8000:8000 myapp:1.0   # app connects to host "db"
```

`-p 8080:80` = **host port : container port**. Inside a container, `localhost` means the container itself, not the host.

8.8 Docker Compose

```
# compose.yaml
services:
  web:
    build: .
    ports:
      - "8000:8000"
    env_file: .env              # keep .env out of Git
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: app
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
volumes:
  pgdata:
```

Command	What it does
`docker compose up -d`	Build (if needed) and start everything
`docker compose ps` / `logs -f web`	Status / logs
`docker compose down`	Stop and remove (add `-v` to delete volumes)
`docker compose build`	Rebuild images

8.9 Container security

- Small, trusted, pinned base images (`python:3.12-slim`, distroless).
- Run as non-root (`USER`), read-only filesystem where possible, drop extra Linux capabilities.

- No secrets in images or Dockerfiles. Pass them at runtime (env from a secrets manager, Docker/K8s secrets).
- Scan images: `trivy image myapp:1.0`. Fix HIGH/CRITICAL findings by updating base images and packages.
- Limit resources: `--memory 512m --cpus 1`.

8.10 Container troubleshooting

Symptom	Check
Container exits immediately	`docker ps -a` (exit code) → `docker logs <name>` → is `CMD` correct?
Keeps restarting	`docker logs`, `docker inspect` (RestartCount, OOMKilled), missing env/config
Exit code 137	Killed, usually out of memory (OOMKilled) → raise the limit or fix the leak
Exit code 1	Application error → read the logs
Can't reach the app	Correct `-p` mapping? App listens on `0.0.0.0`, not `127.0.0.1`? Firewall?
Can't reach another container	Same user-defined network? Using the container name as host? Right port?
Image too big	Slim base, multi-stage, `.dockerignore`, clean caches in the same `RUN`

8.11 More Docker commands and the container ecosystem

Command	What it does
`docker cp web:/etc/nginx/nginx.conf .`	Copy files out of (or into) a container
`docker save -o app.tar myapp:1.0` / `docker load -i app.tar`	Export/import an image as a file
`docker history myapp:1.0`	Layers and their sizes
`docker build --no-cache -t myapp .`	Rebuild without the cache
`docker buildx build --platform linux/amd64,linux/arm64 -t user/app:1.0 --push .`	Multi-architecture build (e.g. for AWS Graviton ARM)
`docker run --rm --env-file .env --restart unless-stopped ...`	Auto-delete on exit / load env file / restart policy
`docker inspect -f '{{.State.ExitCode}}' web`	Pick one field from inspect

Ecosystem: images follow the **OCI** standard, so they run anywhere. Kubernetes uses **containerd** (not Docker) to run containers. **Podman** is a daemonless Docker alternative with the same commands. **Layer order rule:** put things that change rarely (base image, dependencies) first and code last, so rebuilds reuse the cache.

REMEMBER

- Port mapping is **-p HOST:CONTAINER**.
- Dockerfile order: **base → dependencies → code** (code changes most, so it goes last).
- Exit code **137** = killed (usually out of memory). Exit **1** = application error → read the logs.
- Inside a container, `localhost` means the container itself.